

PowerUp Practice — Rebuild B4F Hub's React Architecture

Goal

On Saturday we covered Sessions 01–04 in about three hours, using the real B4F Hub project and its real Express server. You saw every main idea once.

In this practice you **rebuild the important React architecture of B4F Hub yourself**, using the **same real Express API** you saw in class. There is no new product and no new data — the point is repetition and understanding.

You start from B4F Hub exactly as it was at the **start of Session 01**, and you finish with the same architecture the main class had at the **end of Session 04** (plus the Session 04 homework):

- pages and URLs with **React Router**
- notifications shared with **Context**
- saved and recently viewed opportunities in **Redux Toolkit** — two slices
- **selectors**, including selectors that read both slices
- saved data that survives a reload with **localStorage**

Plan for several sittings. Most students need about **10–14 hours** in total. A good rhythm is 4–5 sittings of 2–3 hours (see "Suggested plan" at the end).

What you already have

Folder	What it is	What you do with it
<code>server/</code>	The real B4F Hub Express server — the same one from class. It is already built for you.	Run it. Do not change it.
<code>practice-client/</code>	The real B4F Hub React app as it was at the start of Session 01 : no Router, no Context, no Redux. It already fetches real data from the server.	This is where you work.
<code>client/</code>	The real B4F Hub React app at the end of Session 04 .	Read it only when you are stuck , after you have tried yourself.
<code>docs/session-01 ... docs/session-04</code>	The full lesson material.	Review the matching session before each part.

Start the server and your practice client

You need **two terminals**, both started from the repository root.

Terminal 1 — the server (port 3001). The server is already built for you. Run it and leave it running:

BASH

```
cd server
npm install
npm start
```

You should see `B4F Hub local API` running at `http://localhost:3001`.

Terminal 2 — your practice client (port 5173):

BASH

```
cd practice-client
npm install
npm run dev
```

Open `http://localhost:5173`. You should see the Community and Opportunities panels with real data. If you see loading errors instead, check that Terminal 1 is still running.

Only run one client at a time on port 5173. If `client/` is also running, Vite picks the next free port (for example 5174) — always open the address Vite prints.

How data gets to your screen

This is the same flow you saw in class. Your React code never reads the server's files — it asks the server over HTTP, and the server answers with JSON.

TEXT

```
Browser
  ↓
React component      (for example the Opportunities page)
  ↓ fetch()          (inside fetchOpportunities() in src/api.ts)
/api/opportunities
  ↓                  (Vite forwards every /api request to port 3001)
Express server        (server/index.js — already built for you)
  ↓
JSON data             (an array of 20 opportunities)
  ↓
React state           (useState in the component)
  ↓
UI                   (the opportunity cards)
```

`practice-client/src/api.ts` already contains one function for each request. Use these functions — do not write new `fetch` calls for things they already do.

The real API (already built)

Request	api.ts function	Returns
GET /api/opportunities	fetchOpportunities()	Every opportunity
PATCH /api/opportunities/:id with { applied: true }	applyToOpportunity(id)	The updated opportunity
GET /api/posts	fetchPosts()	Every post, newest first
POST /api/posts with { content, category }	createPost(content, category)	The new post
PATCH /api/posts/:id with { liked }	setPostLiked(id, liked)	The updated post

There is no "get one opportunity" request yet. To show one opportunity, do what class did: fetch the full list with `fetchOpportunities()` and find the one whose `id` matches the URL. Opportunity ids start at 101 .

Where the data lives

Data	Lives in	Survives a browser reload?	Survives a server restart?
Opportunities, posts, "Applied", likes	The server's memory	Yes	No — the server resets to its seed data
Saved opportunity ids	Redux , copied into <code>localStorage</code> by your code	Yes (after CORE)	Yes — they are in the browser, not the server
Recently viewed opportunities	Redux , copied into <code>localStorage</code> by your code	Yes (after CHALLENGE)	Yes
Notifications (toasts)	Context	No — on purpose	No

The app you are building

Routes

URL	Page
/	Home — a welcome and links to Community and Opportunities
/community	The Community panel (posts)
/opportunities	The Opportunities panel (cards, filters, saved, recently viewed)
/opportunities/:id	One opportunity, full details
* (anything else)	"Page not found"

The Navbar shows **Home**, **Community** and **Opportunities**, and the Opportunities link shows how many opportunities are saved.

Who owns which state

State	Tool	Why there
Notifications queue	Context — <code>NotificationContext</code>	App-wide UI messages. Many components <i>send</i> them, one component <i>shows</i> them. No business rules.
Saved opportunity ids	Redux — <code>savedOpportunities</code> slice	Shared app data changed and read from several pages.
Recently viewed opportunities	Redux — <code>recentlyViewed</code> slice	Shared app data with rules (no duplicates, newest first, maximum 5).
The opportunities list, loading and error flags	Local <code>useState</code> in the page that fetches	Comes from the server and belongs to the page that loads it.
Search text and filters	Local <code>useState</code>	Only one panel cares about them.

Context and Redux are **different tools**. Each piece of state lives in exactly one place. If you notice the same data in two places, stop and fix it.

The Redux state you should end up with

TEXT

```
store
├── savedOpportunities
│   ├── savedIds: number[]           e.g. [104, 101]
│   └── recentlyViewed
│       └── entries: { id, title }[]  newest first, max 5
```

Redux state must be plain data. The starter keeps saved ids in a `Set` — in Redux, use an **array** of ids instead (Session 03 explains why).

Difficulty levels

Build the levels **in order** — each one uses what the level before it built.

- **CORE — required for everyone.** Sessions 01–04 as taught in class.
- **STRETCH — expected.** Strongly recommended for everyone, required if you want to call the practice "complete".
- **CHALLENGE — go further.** The Session 04 homework, done after STRETCH works.

CORE (required)

C1. React Router — pages and navigation (Session 01)

1. Install React Router, the same version as class — inside `practice-client/`:

```
npm install "react-router-dom@^7.18.4"
```

1. Wrap the app in `BrowserRouter`.
2. Create a `src/pages/` folder with `HomePage`, `CommunityPage` and `OpportunitiesPage`. The Community page shows the existing `CommunitySection`; the Opportunities page shows the existing `OpportunitiesSection`.
3. In `App.tsx`, replace the two panels with `Routes` and `Route` for `/`, `/community` and `/opportunities`. `Navbar` and `Footer` stay on every page.
4. Add `NavLink`s to the `Navbar` for Home, Community and Opportunities, with an active style. Make sure **Home** is not active on every page.
5. The Home page links to Community and Opportunities with `Link` (never `<a href>`).

C2. The opportunity detail page (Session 01)

1. Add the route `/opportunities/:id` and an `OpportunityDetailPage`.
2. Read the id with `useParams()`. It is a **string**; opportunity ids are **numbers**.
3. Load the data the way class did: call `fetchOpportunities()` and find the matching opportunity. Show a loading message while it loads, and an error message with a retry button if the request fails (the starter already has `LoadingMessage` and `ErrorMessage`).
4. Show everything about the opportunity, including description and deadline, and an **Apply** button that uses `applyToOpportunity(id)`.
5. If no opportunity has that id (try `/opportunities/999`), show a clear message and a `Link` back to `/opportunities`. The page must not crash.
6. Add a **Back** button that returns to the previous page with `useNavigate`.
7. On each card, replace "View details" with a `Link` to `/opportunities/:id`.
8. Remove what the URL now does for you: the expand/collapse details state, the hand-made history "stack", and the `BackToPreviousOpportunity` component.

C3. 404 page (Session 01 homework)

Add a "Page not found" page for any address that is not one of the real pages, with a `Link` back to Home.

C4. NotificationContext (Session 02)

Today, `App.tsx` owns the notifications array and passes `onNotify` down through props. Replace that with `Context`:

1. Create `src/context/NotificationContext.tsx` with `createContext`, a `NotificationProvider` that owns the notifications state, and a custom hook `useNotify()` that throws a clear error when used outside the provider.
2. The context gives: the notifications list, `notify(message, tone)`, and `dismissFront()`.
3. Wrap the app in `NotificationProvider`.
4. `CommunitySection`, `OpportunitiesSection` and `OpportunityDetailPage` call `notify` through `useNotify()`. `ToastQueue` reads the list and dismisses through `useNotify()`.
5. Remove every `onNotify` prop and the notifications state from `App.tsx`.

Test: publish a post, apply to an opportunity on the list **and** on the detail page — every action still shows its toast, in order.

C5. Redux — the `savedOpportunities` slice (Sessions 03–04)

1. Install Redux Toolkit and React Redux, the same versions as class:

BASH

```
npm install "@reduxjs/toolkit@2.12.0" "react-redux@9.3.0"
```

1. Create `src/store/savedOpportunitiesSlice.ts` with `createSlice`. State: `savedIds: number[]`. Actions: `toggleSaved(id)` and `clearAllSaved()`.
2. Create the store with `configureStore` in `src/store/store.ts` and export the `RootState` and `AppDispatch` types.
3. Create typed hooks `useAppDispatch` and `useAppSelector` in `src/store/hooks.ts`, and use them everywhere instead of the plain React Redux hooks.
4. Wrap the app in the React Redux `Provider`.
5. Add named selectors next to the slice: `selectSavedIds`, `selectSavedCount`, and `selectIsSaved(state, id)`.
6. Each `OpportunityCard` reads its own saved state with `selectIsSaved` and dispatches `toggleSaved` itself. Remove the old `Set` state and the `savedIds` / `onToggleSaved` props from `OpportunitiesSection`, `OpportunityList` and `OpportunityCard`.
7. The "Saved only" filter keeps working, reading the saved ids from the store.
8. The Navbar's Opportunities link shows a badge with the saved count (only when it is above 0).
9. A **"Clear saved (N)"** button on the Opportunities panel, shown only when something is saved.

C6. Persist saved opportunities (Session 04)

1. A storage-key constant, `SAVED_OPPORTUNITY_IDS_KEY = "savedOpportunityIds"`.
2. A loader function: read the key; if nothing is stored, or `JSON.parse` throws, return `[]`.
3. Use the loader for the slice's `initialState`.
4. In `store.ts`, **one** `store.subscribe(...)` callback writes `savedIds` to `localStorage` with `JSON.stringify` after every change.

Test: save two opportunities → reload → both still saved, badge still right.

Remember: Redux keeps state **in memory only**. The persistence is your own code *around* Redux — "Redux persists the state" is the wrong way to say it.

STRETCH (expected)

S1. Redux — the `recentlyViewed` slice (Sessions 02–03 homework)

1. Create `src/store/recentlyViewedSlice.ts`. State: `entries`, an array of `{ id, title }`.
2. Reducers:
 - `recordView(entry)` — newest first, **no duplicates** (viewing again moves it to the front), at most **5** entries;

- `removeRecentlyViewed(id)` — removes one entry;
 - `clearRecentlyViewed()` — empties the list.
3. Register it in the **same** store under the key `recentlyViewed` . Do not create a second store.
 4. `OpportunityDetailPage` dispatches `recordView` **only when a matching opportunity was found** — never for a bad id.
 5. Selectors next to the slice: `selectRecentlyViewed` , `selectRecentlyViewedExcluding(state, excludeId?)` , and `selectRecentlyViewedCount` .

S2. The "Recently viewed" strip

1. A `RecentlyViewedList` component that shows the entries as clickable chips (`Link` to each opportunity), or nothing at all when the list is empty.
2. Each chip has a small × remove button with an `aria-label` that names the opportunity. Clicking × must not navigate.
3. A **"Clear (N)"** button, using `selectRecentlyViewedCount` .
4. Show the strip on the Opportunities panel **and** on the detail page. On the detail page it must **exclude** the opportunity you are looking at — use `selectRecentlyViewedExcluding` , not a filter inside the component.

S3. Cross-slice selectors and the Activity summary (Session 04)

1. Create `src/store/selectors.ts` for selectors that read **both** slices. Each one reuses the slices' own named selectors — never `state.savedOpportunities.savedIds` by hand.
2. `selectTotalActivityCount(state)` — saved count + recently viewed count.
3. `selectHasAnyActivity(state)` — whether that total is above 0.
4. On the Opportunities panel, show **"Activity: N"**.
5. Add a **"Reset activity"** button, shown only when there is activity, that clears **both** slices from **one** click handler (two dispatches).

After STRETCH, no component should contain `state.savedOpportunities` or `state.recentlyViewed` — components only use selectors.

CHALLENGE (go further)

These are the Session 04 homework tasks. Read `docs/session-04/HOMEWORK-EN.pdf` for more detail.

X1. Recently viewed, not saved yet

1. Add `selectRecentlyViewedNotSaved(state)` to `selectors.ts` : the recently viewed entries whose id is **not** in the saved ids. Reuse `selectSavedIds` and `selectRecentlyViewed` .
2. In `RecentlyViewedList` , show a short note such as "2 of these aren't saved yet." It never shows 0 — it disappears when nothing is unsaved.

X2. Save all

1. Next to the note, a **"Save all"** button that saves every recently viewed, not-yet-saved opportunity in one click.

2. Dispatch the existing `toggleSaved` **once per entry**, with `.forEach()`.
3. Reload afterwards: everything stays saved — and you wrote no new persistence code. Be ready to explain why.

X3. Persist recently viewed too

1. A **second** key with its own name: `RECENTLY_VIEWED_ENTRIES_KEY = "recentlyViewedEntries"`.
2. A loader with the same try/catch-to- `[]` fallback, used for the slice's `initialState`.
3. **Extend** the existing `store.subscribe` callback to also write `recentlyViewed.entries`. Do **not** add a second `store.subscribe` call.

Rules

Use everything from PowerUp and Sessions 01–04: components, props, `useState`, `useEffect`, the existing `api.ts` functions, `BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink` (with `end`), `useParams`, `useNavigate`, `createContext`, a `Provider`, `useContext` inside a custom hook, `configureStore`, `createSlice`, `Provider`, `useAppDispatch` / `useAppSelector`, named selector functions, `localStorage`, `JSON.stringify` / `JSON.parse`, `store.subscribe`, and array methods (`.filter()`, `.find()`, `.forEach()`, `.includes()`, `.map()`, `.slice()`).

Do not use (not covered yet): `Redux Persist`, `createSelector` / `reselect`, `createAsyncThunk` or any thunk, custom middleware, `extraReducers`, `RTK Query`, `React Query`, `createEntityAdapter`, `useReducer`, nested or layout routes, `useSearchParams`, authentication.

Do not change `server/`. It is the shared data source for this practice. Do not add new endpoints, do not add mock data, and do not copy the data into the client — every opportunity and post comes from the real server through `api.ts`.

Do not change `client/` or `docs/`. Work only inside `practice-client/`.

A console warning you may see

When a selector returns a new array (for example `selectRecentlyViewedExcluding`), React Redux may print a **yellow warning** in the browser console: *"Selector ... returned a different result when called with the same parameters."* It is a development-only performance hint, not an error. The same warning appears in `client/`. You can ignore it here.

Test your app

Keep the server running in Terminal 1. Each line should be true.

N o.	Do this	Expected
1	Open <code>/</code> , <code>/community</code> , <code>/opportunities</code> with the Navbar	Right page each time, right link active, Home not always active
2	Open <code>/opportunities</code>	20 real opportunities from the server

N o.	Do this	Expected
3	Stop the server (Ctrl+C), reload <code>/opportunities</code>	An error message with a retry button — no crash. Start the server again, click retry: the list loads
4	Click "View details" on a card	<code>/opportunities/<id></code> with the full details
5	Open <code>/opportunities/999</code>	"Not found" message with a link back — no crash
6	Open <code>/something-wrong</code>	Your "Page not found" page
7	Publish a post; apply on a card; apply on a detail page	A toast for each, in order
8	Save two opportunities	Stars filled, badge shows 2, "Saved only" shows just those 2
9	Reload the page	Still saved, badge still 2
10	Apply to one, then restart the server (Ctrl+C, <code>npm start</code>) and reload	"Applied" is gone (server memory reset) but saved stays (browser <code>localStorage</code>)
11	(STRETCH) Open 7 different opportunities, go back to <code>/opportunities</code>	5 chips, the last one opened first, no duplicates
12	(STRETCH) On a detail page	The strip does not show the opportunity you are viewing
13	(STRETCH) Click × on one chip; then "Reset activity"	One entry removed; then saved and recently viewed both empty, Activity 0
14	(CHALLENGE) View 3 unsaved opportunities, click "Save all", reload	All saved, the note is gone
15	(CHALLENGE) View 3 opportunities, reload	Same recently viewed list, same order

Also run, inside `practice-client/`:

BASH

```
npm run build
npm run lint
```

Both must finish with **0 errors**.

Explain it in your own words

Create `practice-client/NOTES.md` and answer each question in 3–6 sentences:

1. Follow one opportunity from the server to the screen: which function calls `fetch`, which URL, who answers, and where the result is stored in React.
2. Why are notifications in Context, but saved opportunities in Redux?
3. In test 10, "Applied" disappeared but "saved" stayed. Why?
4. What does `store.subscribe` do in your app, and why is that not "Redux persisting the state"?
5. Pick one selector in `selectors.ts`. Why is it better there than as a filter inside a component?

Suggested plan

#	Goal	Review first
1	C1–C3: Router, pages, Navbar, detail page, 404	docs/session-01
2	C4: NotificationContext	docs/session-02
3	C5–C6: savedOpportunities slice, typed hooks, persistence	docs/session-03 , docs/session-04
4	S1–S3: recentlyViewed slice, the strip, cross-slice selectors	docs/session-02 homework, docs/session-03 , docs/session-04
5	CHALLENGE, the test table, NOTES.md , build and lint	docs/session-04 homework

Commit at the end of every sitting with a message that says what you finished, for example Practice: Router pages and detail page .

Submitting

1. Your instructor will create your own branch, named `student-<your-github-username>` in lowercase, and tell you when it is ready. Work only on that branch.
2. Commit often and push to **your own branch**. Never push to `main` .
3. Your branch must contain your work inside `practice-client/` and `practice-client/NOTES.md` .
4. When you finish, tell your instructor which levels you completed (CORE, STRETCH, CHALLENGE).

Done when

- Every CORE item works, and test lines 1–10 pass.
- STRETCH is complete for a "complete" practice.
- Notifications live only in Context; saved and recently viewed live only in Redux.
- All data still comes from the real server through `api.ts` ; `server/` is unchanged.
- `npm run build` and `npm run lint` finish with 0 errors.
- `NOTES.md` answers all five questions in your own words.